Residual Connections

Hyoung Joon, Kim

June 3, 2026

Contents

1 Introduction

We introduce **residual network**, which enables us to use deep networks, with **batch normalization**. Batch normalization is explained in the other document.

2 Main Ideas

The residual network was introduced to solve the vanishing gradient problem. Ordinary sequential network, without residual connection, the autocorrelation function of the gradient shows that nearby gradients are no so correlated as the layer gets deeper. This means that a subtle change in gradient value leads to tremendous change in the output result. This is termed **shattered gradients** phenomenon.

2.1 Residual Connection

Residual connection is to simply make the network to learn the residual instead of the objective itself. To accomplish this, we just add the input to the output value.

For example, let's assume that we have following network layer:

$$\mathbf{h}_1 = \mathbf{f}_1[\mathbf{x}] \tag{1}$$
$$\mathbf{h}_2 = \mathbf{f}_2[\mathbf{h}_1] \tag{2}$$
$$\mathbf{y} = \mathbf{f}_3[\mathbf{h}_2] \tag{3}$$

This is the ordinary sequential network. Now, we add the input to the output.

$$\mathbf{h}_1 = \mathbf{x} + \mathbf{f}_1[\mathbf{x}] \tag{4}$$
$$\mathbf{h}_2 = \mathbf{h}_1 + \mathbf{f}_2[\mathbf{h}_1] \tag{5}$$
$$\mathbf{y} = \mathbf{h}_2 + \mathbf{f}_3[\mathbf{h}_2] \tag{6}$$

We can interpret the residual connection in two different ways:

1. Ensemble: If we expand the network, we can see that the whole network is some kind of ensemble of small networks:

$$\mathbf{y} = \mathbf{x} + \mathbf{f}_1[\mathbf{x}] \tag{7}$$
$$+ \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] \tag{8}$$
$$+ \mathbf{f}_3[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]]] \tag{9}$$

2. Various Connection from input to output: Since there are various connection from input to output, the vanishing gradient problem is relaxed. You can see that the gradient of $\mathbf{y}$ about $\mathbf{h}_3$ is $\mathbf{I} + \partial\mathbf{f}_4$, and the $\mathbf{I}$ prevents the vanishing gradient problem.

2.1.1 Order of Operations

If we apply ReLU at the end of the network, the residual connection will always add the positive value to the input value. To prevent this, we place ReLU at the front of the network and then apply residual connection. But, the ReLU ignores all negative values, so we first apply linear layer, and then apply ReLU for the first residual block, and apply ReLU first for other residual blocks.

2.1.2 Matching the Dimensions

The residual connection is possible only when the dimensions of the input and output matches. If the dimensions are different, we can apply projection operation and make residual connection.

2.2 Exploding Gradients

The residual connection solves vanishing gradient problem, but cannot solve exploding gradient problem. Even the He initialization cannot resolve the exploding gradient problem in residual network. We use batch normalization to deal with this problem.

The benefits of batch normalization with residual network is presented.

1. **Stable Forward Propagation**: Residual network without batch normalization has variance of the hidden units increasing exponentialy. With batch normalization, the variance increases linearly, and even this can be resolved by tuning the mean and standard deviation used in batch normalization.
2. **Higher Learning Rates**: Empirical studies and theory both show that batch normalization smoothens the loss surface. This means we can use higher learning rates.
3. **Regularization**: Since the batch normalization uses batch dataset, this injects noise to the network (since the statistics are different for each batches), which has the effect of regularization.

3 Extensions of Residual Connection

3.1 Neural ODE

The residual connection follows the equation when applied: $\mathbf{z}^{(t+1)} = z^{(t)} + \mathbf{f}(\mathbf{z}^{(t)}, \mathbf{w})$. If we increase the number of residual layers infinitely, we can represent the vector $\mathbf{z}^{(t)}$ as a continuous function of t, satisfying following equation:

$$\frac{d\mathbf{z}(t)}{dt} = \mathbf{f}(\mathbf{z}(t), \mathbf{w})$$

We can evaluate this by using ODE solver, such as RK method or Euler's forward integration method.

3.1.1 Training Neural ODE

Suppose that we are given the input vector $\mathbf{z}(0)$, and the loss functions depend on $\mathbf{z}(T)$, the output vector. To apply backpropagation to neural ODE, we first define a quantity called **adjoint**.

Definition 3.1.1 (Adjoint)

Let $\mathbf{z}(t)$ denote the continuous layer given as $\frac{d\mathbf{z}(t)}{dt} = \mathbf{f}(\mathbf{z}(t), \mathbf{w})$, and let L denote the loss function. We define **adjoint** $\mathbf{a}(t)$ as

$$\mathbf{a}(t) = \frac{dL}{d\mathbf{z}(t)}$$

Notice that $\mathbf{a}(T)$ corresponds to the usual derivative of the loss with respect to the output vector. The adjoint satisfies its own differential equation:

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)\nabla_{\mathbf{z}}\mathbf{f}(\mathbf{z}(t), \mathbf{w})$$

Proof. The following holds for arbitrary dt: $\mathbf{z}(t+dt) = \mathbf{z}(t) + \mathbf{f}(\mathbf{z}(t), \mathbf{w})dt + \boldsymbol{\epsilon}_z$ where $\lim_{dt\to 0} \frac{\|\boldsymbol{\epsilon}_z\|}{dt} = 0$. Similar equation hols for the adjoint $\mathbf{a}(t)$, $\mathbf{a}(t+dt) = \mathbf{a}(t) + \frac{d\mathbf{a}(t)}{dt}dt + \boldsymbol{\epsilon}_a$ where $\lim_{dt\to 0} \frac{\|\boldsymbol{\epsilon}_a\|}{dt} = 0$. The following holds due to the chain rule:

$$\mathbf{a}(t) = \frac{dL}{d\mathbf{z}(t)} = \frac{dL}{d\mathbf{z}(t+dt)}\frac{d\mathbf{z}(t+dt)}{d\mathbf{z}(t)}$$

Note that $\mathbf{a}(t+dt) = \frac{dL}{d\mathbf{z}(t+dt)}$. Then,

$$\begin{aligned}\mathbf{a}(t) &= \mathbf{a}(t+dt)\frac{d\mathbf{z}(t+dt)}{d\mathbf{z}(t)} \\ &= \mathbf{a}(t+dt)\left(\mathbf{I} + \frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{z}(t)}dt + \frac{d\boldsymbol{\epsilon}_z}{d\mathbf{z}(t)}\right)\end{aligned}$$

Using the equation $\mathbf{a}(t+dt) = \mathbf{a}(t) + \frac{d\mathbf{a}(t)}{dt} + \boldsymbol{\epsilon}_a$, we get

$$\frac{d\mathbf{a}(t)}{dt} + \frac{\boldsymbol{\epsilon}_a}{dt} = -\mathbf{a}(t+dt)\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{z}(t)} - \mathbf{a}(t+dt)\frac{d\boldsymbol{\epsilon}_z}{d\mathbf{z}(t)}\frac{1}{dt}$$

As $dt \to 0$, the equation goes to

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{z}(t)}$$

□

Now we evaluate the derivatives of the loss with respect to network parameters.

Theorem 3.1.1 (Gradient of Loss Function about Parameter)

Let $\mathbf{a}(t)$ be the adjoint and let L be the loss function. Then, the following equation holds;

$$\frac{dL}{d\mathbf{w}} = \int_{t_0}^{T} \mathbf{a}(t)\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{w}}dt$$

Proof. Following holds since $\frac{d\mathbf{z}(t)}{dt} = \mathbf{f}(\mathbf{z}(t), \mathbf{w})$

$$\mathcal{L} = L(\mathbf{z}(T)) - \int_{t_0}^{T} \mathbf{a}(t)\left(\frac{d\mathbf{z}(t)}{dt} - \mathbf{f}(\mathbf{z}(t), \mathbf{w})\right)dt.$$

We use integration by parts and derive the following equation:

$$\int_{t_0}^{T} \mathbf{a}(t)\frac{d\mathbf{z}(t)}{dt}dt = \left[\mathbf{a}(t)\mathbf{z}(t)\right]_{t_0}^{T} - \int_{t_0}^{T} \frac{d\mathbf{a}(t)}{dt}\mathbf{z}(t)dt.$$

We substitute this at the original equation and obtain

$$\mathcal{L} = L(\mathbf{z}(T)) - \mathbf{a}(T)\mathbf{z}(T) + \mathbf{a}(t_0)\mathbf{z}(t_0) + \int_{t_0}^{T} \left(\frac{d\mathbf{a}(t)}{dt}\mathbf{z}(t) + \mathbf{a}(t)\mathbf{f}(\mathbf{z}(t), \mathbf{w}) \right) dt.$$

We now differentiate the equation about $\mathbf{w}$.

$$\frac{d\mathcal{L}}{d\mathbf{w}} = \left(\frac{dL}{d\mathbf{z}(T)} - \mathbf{a}(T) \right) \frac{d\mathbf{z}(T)}{d\mathbf{w}} + \mathbf{a}(t_0)\frac{d\mathbf{z}(t_0)}{d\mathbf{w}} + \int_{t_0}^{T} \left\{ \left(\frac{d\mathbf{a}(t)}{dt} + \mathbf{a}(t)\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{z}(t)} \right) \frac{d\mathbf{z}(t)}{d\mathbf{w}} + \mathbf{a}(t)\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{w}} \right\} dt$$

Since these equation are satisfied about any $\mathbf{a}(t)$, we can substitute $\mathbf{a}(t)$ with the adjoint. Then, we get

$$\frac{dL}{d\mathbf{w}} = \frac{d\mathcal{L}}{d\mathbf{w}} = \int_{t_0}^{T} \mathbf{a}(t)\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{w}} dt$$

□

So, actually the adjoint is originaly created from the idea of Lagrange multiplier.

The benefit of neural ODE using adjoint method is that there is no need to store the intermediate results of the forward propagation, which leads to constant memory cost.

Algorithm 1: Neural ODE training algorithm

Input : Neural ODE $\frac{d\mathbf{z}(t)}{dt} = \mathbf{f}(\mathbf{z}(t), \mathbf{w})$
loss function $L(\cdot, \cdot)$
input data $\mathbf{z}(0)$
correct answer $\mathbf{z}'$
learning rate η

Output: empty

`// forward pass`

$\mathbf{z}(T) \leftarrow \text{ODE Solver}_{0 \to T}[\frac{d\mathbf{z}(t)}{dt} = \mathbf{f}(\mathbf{z}(t), \mathbf{w})](\mathbf{z}(0))$

$L \leftarrow L(\mathbf{z}(T), \mathbf{z}')$

`// backward pass`

$$\begin{bmatrix} \mathbf{z}(0) \\ \mathbf{a}(0) \\ \mathbf{a_w}(0) \end{bmatrix} \leftarrow \text{ODE Solver}_{T \to 0} \left[\frac{d}{dt} \begin{bmatrix} \mathbf{z}(t) \\ \mathbf{a}(t) \\ \mathbf{a_w}(t) \end{bmatrix} = \begin{bmatrix} \mathbf{f}(\mathbf{z}(t), \mathbf{w}) \\ -\mathbf{a}(t)\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{z}(t)} \\ -\mathbf{a}(t)\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{w}} \end{bmatrix} \right] \left(\begin{bmatrix} \mathbf{z}(T) \\ \frac{dL}{d\mathbf{z}(T)} \\ 0 \end{bmatrix} \right)$$

`// Update`

$\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{a_w}(0)$

Since the intermediate values are not saved, we re-calculate the intermediate values during backpropagation.